

Course: PCCCS495: Advanced Programming (OOP) Lab

Term II Project Report

Project Title: Inventory Management System Using DAO Pattern with File Persistence

Submitted by: Subhashree Das

Github Username: das-subho06

Class Roll no.: 64

Enrollment no.: 12024052017066

Department: CSE(IOT CS BT)

Course Instructor: Soumadip Biswas Sir, Susovan Jana Sir, Swagatam Basu Sir.

Semester: 4th (Spring 2026)

IEM Gurukul GitHub: [IEM-Gurukul / term-ii-project-submission-das-subho06](#)

Problem Statement:

Small businesses – retail shops, pharmacies, general stores – typically manage their inventory through paper registers or spreadsheet files. This approach suffers from a range of operational problems: data is frequently lost due to accidental deletion or hardware failure; there is no built-in mechanism to search or filter records quickly; and, critically, there is no automated alert system to warn the owner when a product is running low and needs restocking.

This project addresses these problems by delivering a Java-based desktop application, InvenTrack, that allows a shopkeeper to digitally manage Products, Categories, and Suppliers from a single graphical interface. The system uses the Data Access Object (DAO) design pattern to cleanly separate data-access logic from business logic, and persists all records to local binary files using Java Object Serialization removing any dependency on an external database or internet connection.

The Service layer enforces business rules on every user action, validating inputs such as non-empty names, positive selling prices, non-negative quantities, and referential integrity between entities (a product must reference a valid category and supplier). Features include full CRUD operations for all three entity types, live dashboard statistics, a dedicated Low Stock Alert panel with one-click restock capability, name-based product search, category filtering, sortable table columns, and CSV export of the entire inventory – making the system both practical and maintainable for small-scale use.

System Design

UML Class Diagram

MODEL

Product
- id: String - name: String - description: String - price: double - costPrice: double - quantity: int - lowStockThreshold: int - categoryId: String - supplierId: String - dateAdded: LocalDate
+ isLowStock(): boolean + getProfitMargin(): double + getStockValue(): double

Category
- id: String - name: String - description: String

Supplier
- id: String - name: String - contactName: String - phone: String - email: String - address: String

DAO LAYER

(interface)
ProductDAO
+ save(p: Product) + update(p: Product) + delete(id: String) + findById(id: String) + findAll(): List<Product> + searchByName(name: String) + findLowStock()

CategoryDAO
(interface)
+ save(c: Category) + update(c: Category) + delete(id: String) + findAll(): List<Category>

ProductDAOImpl
- filePath: String
+ save(p: Product) + update(p: Product) + delete(id: String) + findAll(): List<Product> - loadAll(): List<Product>

CategoryDAOImpl
- filePath: String
+ loadAll(): List<Category>

(interface) SupplierDAO
+ save(s: Supplier)
+ update(s: Supplier)
+ delete(id: String)
+ findAll(): List<Supplier>

SupplierDAOImpl
- filePath: String
- loadAll(): List<Supplier>

SERVICE LAYER

ProductService
- productDAO: ProductDAO
+ addProduct(p: Product)
+ updateProduct(p: Product)
+ deleteProduct(id: String)
+ getAllProducts(): List<Product>
+ searchByName(name: String)
+ getLowStockProducts()

CategoryService
- categoryDAO: CategoryDAO
+ addCategory(c: Category)
+ updateCategory(c: Category)
+ deleteCategory(id: String)
+ getAllCategories(): List

SupplierService
- supplierDAO: SupplierDAO
+ addSupplier(s: Supplier)
+ updateSupplier(s: Supplier)
+ deleteSupplier(id: String)
+ getAllSuppliers(): List

UTILITY

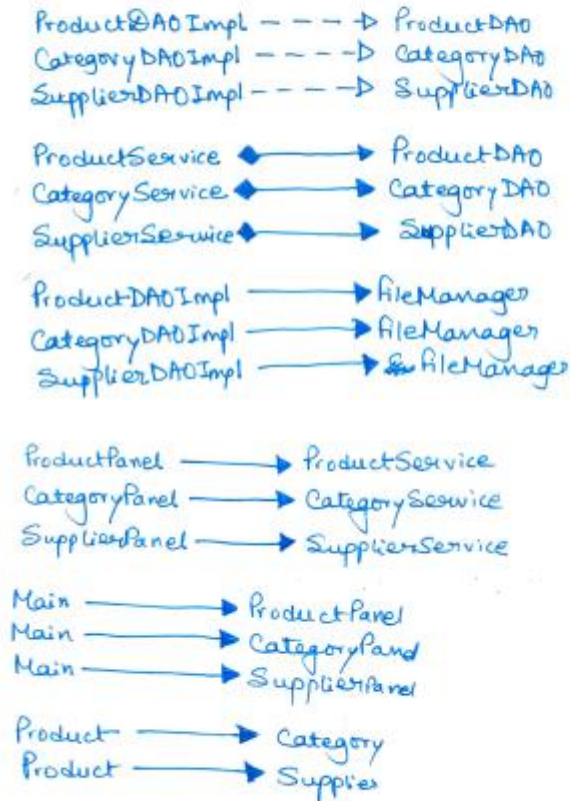
FileManager
+ saveData(list, filePath)
+ loadData(filePath)

CsvExporter
+ exportProducts(list)

UI (SWING)



RELATIONSHIPS



* * * * *

Description of Major Classes:

Class / Interface	Responsibility
Product	Model: stores all product fields; implements Serializable; provides isLowStock() and getProfitMargin() helpers.

Category / Supplier	Lightweight model classes holding reference data used by products.
ProductDAO (interface)	Declares the full CRUD contract: save, update, delete, findById, findAll, findByCategoryId, findLowStock, searchByName, generateNextId.
ProductDAOImpl	Implements ProductDAO using FileManager; loads the list on every operation, mutates it, and writes it back.
ProductService	Validates all business rules before delegating to the DAO; enforces referential integrity; the only layer the UI interacts with.
FileManager	Generic utility using Java Object Serialization to persist any List<T> to a .dat binary file; handles folder creation and I/O exceptions.
MainFrame	Root Swing window; owns a CardLayout container and a sidebar; switches panels and triggers data refresh on navigation.
DashboardPanel	Displays live summary statistics (total products, total value, low-stock count) and a table of low-stock items.
ProductPanel	Full CRUD panel for products; contains search field, category filter combo-box, sortable Jtable, and action buttons.
LowStockPanel	Dedicated alert panel showing quantity vs. threshold with a shortage column and one-click restock dialog.
CsvExporter	Static utility that writes all products to a timestamped CSV file in exports/; handles RFC 4180 comma-escaping.
ValidationException	Custom unchecked exception thrown by all service methods on rule violations; caught by UI panels to display JOptionPane error dialogs.

Package Structure:

com.inventory

```

├─ Main.java           ( Application entry point)
├─ model/
│   ├── Product.java
│   ├── Category.java
│   └─ Supplier.java
├─ dao/
│   ├── interfaces/
│   │   ├── ProductDAO.java
│   │   ├── CategoryDAO.java
│   │   └─ SupplierDAO.java
│   └─ impl/
│       ├── ProductDAOImpl.java
│       ├── CategoryDAOImpl.java
│       └─ SupplierDAOImpl.java
├─ service/
│   ├── ProductService.java
│   ├── CategoryService.java
│   └─ SupplierService.java
├─ ui/
│   ├── AppTheme.java
│   ├── MainFrame.java
│   └─ DashboardPanel.java

```

```

|   ├── ProductPanel.java
|   ├── CategoryPanel.java
|   ├── SupplierPanel.java
|   └── LowStockPanel.java
└── util/
    ├── FileManager.java
    └── CsvExporter.java
└── exception/
    └── ValidationException.java

```

Interaction Flow:

1. The user performs an action in a UI panel (e.g., clicks Add Product after filling the form fields).
2. The panel collects input values and calls the appropriate Service method (e.g., `productService.addProduct(...)`).
3. The Service validates every field; if a rule is violated, a `ValidationException` is thrown and propagated to the UI, which displays an error dialog.
4. If validation passes, the Service constructs the domain object and calls the DAO interface method (e.g., `productDAO.save(product)`).
5. The DAO implementation delegates to `FileManager.saveData()`, which serializes the updated list to `products.dat`.
6. Control returns to the UI, which calls `refreshTable()` to reload and redisplay the updated data.

OOP Concepts Demonstrated

Encapsulation

Where it is used: All three model classes — Product, Category, and Supplier — declare every field as private and expose them exclusively through public getter and setter methods.

Why it is used: This prevents other classes from directly modifying the internal state of a model object in an uncontrolled manner. For example, no external class can set a negative quantity directly; all mutations go through the Service layer, which validates first.

Code Snippet:

Product.java

```

public class Product implements Serializable {

    private String id;
    private String name;
    private double price;
    private int    quantity;
    private int    lowStockThreshold;

    // Derived behaviour encapsulated inside the class
    public boolean isLowStock() {
        return quantity <= lowStockThreshold;
    }

    public double getProfitMargin() {
        if (costPrice == 0) return 0.0;
        return ((price - costPrice) / price) * 100.0;
    }
}

```



```

    }

    public int getQuantity()          { return quantity; }
    public void setQuantity(int qty) { this.quantity = qty; }
}

```

Explanation: All fields are private, so no other class reads or writes them directly. The helper methods `isLowStock()` and `getProfitMargin()` encapsulate derived logic: the `LowStockPanel` and `DashboardPanel` simply call these methods without re-implementing the threshold comparison or margin formula themselves, which eliminates code duplication and centralises business logic in the model.

Abstraction

Where it is used: The DAO interfaces `ProductDAO`, `CategoryDAO`, and `SupplierDAO` declare method signatures — `save()`, `update()`, `delete()`, `findById()`, `findAll()`, `searchByName()`, etc. — without specifying how the data is stored.

Why it is used: The Service layer and UI panels program against the interface contract, not against a specific storage mechanism. If the application were later upgraded from file serialization to a SQLite database, only the impl/ classes would change; every other layer of the system would remain untouched.

Code Snippet:

`ProductDAO.java`

```

public interface ProductDAO {
    void save(Product product);
    void update(Product product);
    void delete(String id);
    Optional<Product> findById(String id);
    List<Product> findAll();
    List<Product> findById(String categoryId);
    List<Product> findLowStock();
    List<Product> searchByName(String keyword);
    String generateNextId();
}

```

Explanation: This interface contains nine method signatures and zero implementation code. `ProductService` holds a reference of type `ProductDAO`, so it calls `productDAO.save(product)` without ever knowing whether data is going to a file, a database, or a network service. The abstraction boundary is clean and total.

Polymorphism

Where it is used: (a) The `ProductService` field `productDAO` is declared as type `ProductDAO` (interface) but instantiated as `ProductDAOImpl` at runtime. (b) A custom `DefaultTableCellRenderer` is overridden in each panel to provide different row-colouring behaviour from the same method signature.

Why it is used: It decouples the calling code from the concrete class. The Service never needs to know which DAO class it is using; it simply calls the interface methods and the JVM dispatches the call to the correct implementation.

Code Snippet:

`ProductService.java`

```

public class ProductService {

```

```
// Declared as the interface type – not the concrete class
private ProductDAO productDAO = new ProductDAOImpl();

public List<Product> getLowStockProducts() {
    // productDAO.findLowStock() dispatches to ProductDAOImpl
    // at runtime – the Service does not know or care which impl
    return productDAO.findLowStock();
}
}
```

Explanation: Because productDAO is typed as ProductDAO (the interface), swapping new ProductDAOImpl() for a hypothetical new DatabaseProductDAOImpl() on the single instantiation line is the only change needed to switch the entire storage layer — no modifications to service logic are required. This directly demonstrates run-time polymorphism and the Open/Closed Principle.

Inheritance

Where it is used: (a) ValidationException extends RuntimeException, inheriting exception-handling infrastructure. (b) All five UI panels — DashboardPanel, ProductPanel, CategoryPanel, SupplierPanel, and LowStockPanel — extend JPanel, inheriting Swing's layout and painting mechanisms.

Why it is used: ValidationException inherits the message-storage, stack-trace capture, and propagation machinery of RuntimeException, requiring only a single constructor. Extending JPanel allows each panel to be added directly to a CardLayout container in MainFrame without any adaptor code.

Code Snippet:

ValidationException.java

```
package com.inventory.exception;
// Extends RuntimeException – inherits getMessage(), printStackTrace(),
// getCause(), and all other exception infrastructure automatically.
Public class ValidationException extends RuntimeException {

    private static final long serialVersionUID = 1L;

    public ValidationException(String message) {
        super(message); // passes message to RuntimeException
    }
}
```

Explanation: The entire class body is just three lines of meaningful code, yet it gains the full RuntimeException behaviour through inheritance. Being unchecked (inheriting from RuntimeException rather than Exception) means callers are not forced to declare it in a throws clause, keeping the service-method signatures clean.

Exception Handling

Where it is used: Every service method validates input and throws ValidationException on rule violation. FileManager wraps all I/O operations in try-with-resources blocks and converts checked IOException to RuntimeException. Main.java registers a global uncaught-exception handler.

Why it is used: Separating validation from the UI layer means that no panel has to contain its own checking logic; the Service is the single source of truth for what is valid. Wrapping checked I/O

exceptions prevents every calling site from having to handle them, since file-I/O failure is a non-recoverable runtime error in this context.

Code Snippet:

FileManager.java

```
public static <T> void saveData(List<T> list, String filePath) {
    ensureDataFolderExists();
    try (
        FileOutputStream fos = new FileOutputStream(filePath);
        ObjectOutputStream oos = new ObjectOutputStream(fos)
    ) {
        oos.writeObject(list);          // streams closed automatically
    } catch (IOException e) {
        // Rethrow as unchecked – callers don't need to handle it
        throw new RuntimeException(
            "Error saving data to file: " + filePath
            + "\nReason: " + e.getMessage());
    }
}
```

Explanation: The try-with-resources construct guarantees that `FileOutputStream` and `ObjectOutputStream` are closed whether or not an exception occurs, preventing resource leaks. Converting `IOException` to `RuntimeException` is a conscious design choice: since the application cannot sensibly continue if it cannot persist data, this is treated as a fatal condition and reported via the global handler in `Main.java`.

Collections and Java Streams

Where it is used: All data is held in `List<T>` collections. Java Streams are used in all three DAO implementations for search, filter, and ID-generation operations.

Why it is used: Collections provide dynamic sizing and convenient iteration. Streams allow declarative, readable transformations without manual loops, improving both correctness and clarity.

Code Snippet:

ProductDAOImpl.java

```
@Override
public List<Product> searchByName(String keyword) {
    String lower = keyword.toLowerCase();
    return loadAll().stream()
        .filter(p -> p.getName().toLowerCase().contains(lower))
        .collect(Collectors.toList());
}

@Override
public String generateNextId() {
    List<Product> list = loadAll();
    if (list.isEmpty()) return "PRD001";
    int max = list.stream()
        .mapToInt(p -> Integer.parseInt(p.getId().substring(3)))
        .max().orElse(0);
    return String.format("PRD%03d", max + 1);
}
```

Explanation: The first method pipelines three stream operations — `stream()`, `filter()`, `collect()` — to produce a filtered sublist without mutating the source. The second uses `mapToInt()` and `max()` to compute the highest numeric suffix and increments it, guaranteeing unique, sequential IDs (PRD001, PRD002, ...) even after deletions in the middle of the sequence.

Implementation Highlights

Generic FileManager Using Type Parameters

`FileManager.saveData()` and `loadData()` are declared as generic methods using the type parameter `<T>`, allowing the same I/O logic to persist Products, Categories, and Suppliers through identical method calls. This eliminates three separate persistence utility classes and demonstrates correct use of Java Generics.

```
// Works for List<Product>, List<Category>, List<Supplier> – same method
public static <T> void saveData(List<T> list, String filePath) {
    try (ObjectOutputStream oos = new ObjectOutputStream(
        new FileOutputStream(filePath))) {
        oos.writeObject(list);
    } catch (IOException e) {
        throw new RuntimeException("Save failed: " + e.getMessage());
    }
}

@SuppressWarnings("unchecked")
public static <T> List<T> loadData(String filePath) {
    File f = new File(filePath);
    if (!f.exists()) return new ArrayList<>();
    try (ObjectInputStream ois = new ObjectInputStream(
        new FileInputStream(f))) {
        return (List<T>) ois.readObject();
    } catch (IOException | ClassNotFoundException e) {
        throw new RuntimeException("Load failed: " + e.getMessage());
    }
}
```

Justification: The unchecked cast on `readObject()` is unavoidable because the JVM erases generic type information at runtime (type erasure). The `@SuppressWarnings("unchecked")` annotation documents this known, safe cast explicitly rather than silently suppressing the warning.

Input Validation with Referential Integrity

`ProductService.addProduct()` performs six validation checks before creating a product, including foreign-key-style lookups that ensure the referenced category and supplier actually exist. This prevents orphaned records without requiring a relational database.

```
Public void addProduct(String name, String description,
    double price, double costPrice, int quantity, int threshold,
    String categoryId, String supplierId) {
```

```

    if (name == null || name.trim().isEmpty())
        throw new ValidationException("Name cannot be empty.");
    if (price <= 0)
        throw new ValidationException("Price must be > 0. Got: " + price);
    if (costPrice < 0)
        throw new ValidationException("Cost price cannot be negative.");
    if (quantity < 0)
        throw new ValidationException("Quantity cannot be negative.");
    if (!categoryDAO.findById(categoryId).isPresent())
        throw new ValidationException("Category " + categoryId + " not found.");
    if (!supplierDAO.findById(supplierId).isPresent())
        throw new ValidationException("Supplier " + supplierId + " not found.");

    productDAO.save(new Product(productDAO.generateNextId(), name.trim(), ...));
}

```

Justification: Guard-clause style validation is used: each rule is checked independently and throws immediately on failure, keeping the happy-path code at the bottom of the method and making the validation logic easy to audit one rule at a time.

Low Stock Detection Encapsulated in the Model

Rather than scattering threshold comparisons across multiple panels, the logic is encapsulated directly in the Product model. Every panel that needs to flag low stock simply calls `product.isLowStock()`, guaranteeing a single, consistent definition of “low stock” throughout the application.

```

// Product.java – single source of truth for the low-stock rule
public boolean isLowStock() {
    return quantity <= lowStockThreshold;
}

// ProductDAOImpl.java – uses method reference for clarity
public List<Product> findLowStock() {
    return loadAll().stream()
        .filter(Product::isLowStock)
        .collect(Collectors.toList());
}

```

Justification: The method reference `Product::isLowStock` is functionally equivalent to `p -> p.isLowStock()` but more concise and communicates intent clearly. The `LowStockPanel` computes the shortage column as (threshold – quantity) for each item, giving the shopkeeper a precise restock quantity at a glance.

CSV Export

`CsvExporter` generates a timestamped CSV file in the `exports/` folder. It correctly handles the edge case where product names or descriptions contain commas or double-quote characters.

```

private static String escape(String value) {
    if (value == null) return "";

```

```
// Step 1: double any embedded double-quote characters
String escaped = value.replace("\"", "\\\"");
// Step 2: wrap in quotes if field contains delimiter or whitespace
if (escaped.contains(",") || escaped.contains("\n") || escaped.contains("\r")) {
    return "\"" + escaped + "\"";
}
return escaped;
}
```

Justification: Without this two-step escape, a product named “Laptop, 15 inch” would corrupt the CSV column alignment, and a product description containing a double-quote character would produce invalid CSV. The standard approach of doubling embedded quotes before wrapping is implemented here exactly as specified in RFC 4180.

Testing and Error Handling

Test Cases Considered:

Scenario	Expected Outcome
Add product with valid inputs	Product saved; ID auto-generated as PRD00n; appears in table.
Add product with empty name	ValidationException thrown; dialog displays “Name cannot be empty”.
Add product with price = 0 or negative	ValidationException thrown; dialog displays “Price must be > 0”.
Add product with non-existent category ID	ValidationException thrown; dialog shows category not found.
Update existing product details	DAO updates record in-place; table refreshes with new values.
Delete a product that exists	Product removed from file; table reflects deletion immediately.
Search product by partial name	All matching products returned; non-matching products hidden.
Add product with quantity = 0	Saved successfully; shown as LOW STOCK if threshold > 0.
Restock product via Low Stock panel	New quantity saved; item disappears from low-stock view.
Export inventory to CSV	File created in exports/ with correct column headers and data rows.
Delete category still referenced by products	ValidationException thrown; category not deleted; error shown.

Edge Cases Handled:

- Empty search keyword: Returns the full product list rather than an empty result, preventing a confusing blank screen for the user.
- Duplicate category name: CategoryService.addCategory() scans existing categories with a case-insensitive comparison before saving, preventing duplicates like “electronics” and “Electronics”.
- ID generation after deletion: generateNextId() always finds the current maximum ID and increments it, so IDs are never reused even if intermediate records are deleted.

- data/ folder not present: FileManager.ensureDataFolderExists() creates the folder automatically on first use; the application never crashes on a fresh install.
- Negative low-stock threshold: Validated as non-negative; a threshold of zero effectively disables the low-stock alert for that product.
- CSV fields containing commas or quotes: Handled by the RFC 4180-compliant escape() method in CsvExporter.

Failure Scenarios and Recovery:

- File I/O error on save: FileManager catches IOException and throws RuntimeException; Main.java's global uncaught-exception handler intercepts it and displays an error dialog via JOptionPane, preventing a silent crash.
- Corrupted .dat file: If a .dat file is unreadable (e.g., due to an interrupted write), loadData() throws RuntimeException, which is caught by the global handler; the user is informed and the application continues with an empty list.
- Null or missing optional values: All findById() calls return Optional<T>; callers always check .isPresent() before accessing the value, so NullPointerExceptions on those paths are structurally impossible.
- Concurrent UI updates: All UI construction and updates are dispatched via SwingUtilities.invokeLater(), ensuring they execute on the Event Dispatch Thread and avoiding race conditions.

Git Workflow

Commit History:

The screenshot displays a Git commit history interface. At the top, there's a header 'Commits' with a search bar set to 'main' and filters for 'All users' and 'All time'. Below this, the commits are grouped by date. The first group is for 'Commits on Mar 29, 2026', showing three commits: 'UML Diagram added' (commit 179aa73, 10 hours ago), 'modified README.md file' (commit b8b0ed1, 12 hours ago), and 'added CsvExporter, tried to resolve issue with TableHeader' (commit 242d263, 13 hours ago). The second group is for 'Commits on Mar 28, 2026', showing three commits: 'added dedicated LowStockPanel with restock dialog and shortage column' (commit 6b1d539, yesterday), 'added column sorting to Product, Category, Supplier tables' (commit 13217cc, yesterday), and 'added CategoryPanel, SupplierPanel, update Main.java to launch GUI' (commit 20e56da, yesterday). Each commit entry includes a commit icon, the commit message, the commit hash, the time since commit, and links to view the commit details or the code diff.

Date	Commit Message	Commit Hash	Time Since Commit
Mar 29, 2026	UML Diagram added	179aa73	10 hours ago
	modified README.md file	b8b0ed1	12 hours ago
	added CsvExporter, tried to resolve issue with TableHeader	242d263	13 hours ago
Mar 28, 2026	added dedicated LowStockPanel with restock dialog and shortage column	6b1d539	yesterday
	added column sorting to Product, Category, Supplier tables	13217cc	yesterday
	added CategoryPanel, SupplierPanel, update Main.java to launch GUI	20e56da	yesterday

added CategoryPanel, SupplierPanel, update Main.java to launch GUI	20e5fda	<>
added ProductPanel with full CRUD, search, filter, and restock	3c4220f	<>
added DashboardPanel with stat cards and low-stock table	5b3c35a	<>
Commits on Mar 27, 2026		
add AppTheme constants and MainFrame with sidebar navigation	9dad58a	<>
Commits on Mar 26, 2026		
added service layer with full input validation and business rules	99d50a9	<>
Commits on Mar 22, 2026		
implemented file-based DAO for Product, Category, Supplier	7c654a4	<>
added FileManager	545dac8	<>
added DAO interfaces for Product, Category, Supplier	6db9ceb	<>
Commits on Mar 21, 2026		
add Product, Category, Supplier model classes	ad9cf9a	<>
Commits on Mar 20, 2026		
add deadline	Verified d4b0192	<>
Commits on Mar 2, 2026		
Initial commit	Verified 59939a0	<>

Development Progression

The commit history reflects a deliberate bottom-up development strategy aligned with the DAO pattern's layered architecture:

- Phase 1 (Commits 2–3): The model and interface layers were established first, creating stable contracts for all subsequent work without any implementation dependency. This allowed each subsequent layer to be written against a known, fixed API.
- Phase 2 (Commits 4–5): Persistence infrastructure was built next. FileManager was isolated as its own commit to allow independent testing before the three DAO implementations were added on top of it.
- Phase 3 (Commit 6): The service layer was completed before any UI work began, ensuring that all business rules were functional and testable independently of the graphical interface. This separation made debugging straightforward.
- Phase 4 (Commits 7–12): UI panels were added incrementally — the visual framework first (MainFrame, AppTheme), then the Dashboard, then the most complex panel (Products with full CRUD and search), then the remaining entity panels, then the dedicated LowStock panel.
- Phase 5 (Commits 13–14): Refinements and bonus features — CSV export and column sorting — were added after the core system was stable. This kept the critical path risk-free and ensured that bonus work could not break the core functionality.

Conclusion and Future Scope

Conclusion:

The InvenTrack Inventory Management System successfully demonstrates all six OOP concepts stipulated in the project specification within a cohesive, functional desktop application. Encapsulation is enforced through private fields and public accessors in all model classes. Abstraction is achieved through the DAO interface contracts that completely hide file-I/O details from upper layers. Polymorphism is exercised both at the DAO level (interface references resolved to concrete implementations at runtime) and at the UI level (overridden DefaultTableCellRenderer). Inheritance is used for ValidationException and all five UI panels. Exception handling provides both user-facing validation feedback via JOptionPane dialogs and robust I/O error recovery via the global uncaught-exception handler. Java Collections and Streams power all data retrieval, search, and filtering operations throughout the application.

The DAO design pattern proved to be an excellent architectural choice for this scale of project, enforcing strict separation of concerns and enabling the entire storage mechanism to be replaced without touching any layer above the DAO implementations — a property that directly addresses the instructor’s feedback regarding extensibility. The system fully implements CRUD for all three entities, addresses all five improvement points raised in the APPROVED_MINOR feedback, and adds bonus features including a dedicated Low Stock Alert panel, column sorting, and CSV export.

Future Scope

- Database Integration: Replace FileManager with a JDBC-based DAO implementation targeting SQLite or H2 to support larger inventories without changing the Service or UI layers - directly demonstrating the DAO pattern’s extensibility.
- User Authentication: Implement a login screen with role-based access (Admin vs. Read-Only Viewer) to make the system suitable for multi-user environments.
- Advanced Reporting: Generate graphical reports (bar charts via JfreeChart) for inventory value trends, category distribution, and monthly restock activity.
- Automated Restock Alerts: Integrate JavaMail to send email notifications automatically when any product’s quantity drops below its threshold.
- Barcode / QR Scanning: Add support for scanning product barcodes via a webcam to speed up stock entry and lookup in a physical retail setting.
- Undo / Redo Mechanism: Implement a command-pattern-based undo stack so shopkeepers can reverse accidental deletions or edits without restarting the application.

APPENDIX A — INITIAL PROJECT PROPOSAL

Project Proposal Document

Project Title: Inventory Management System using DAO Pattern and File Handling in Java

Name: Subhashree Das

Department: CSE(IOT CS BT)

Year: 2nd

Section : A

Roll no.: 64

Enrollment no.: 12024052017066

Problem Statement (Max 150 words)

Managing product inventory efficiently is important for small businesses and shops to track available items, update stock levels, and maintain organized records. However, many small stores still rely on manual record-keeping methods such as notebooks or unstructured files, which can lead to errors, misplaced information, and difficulty in updating product data. To address this problem, an Inventory Management System will be developed using Java Object-Oriented Programming concepts and the DAO (Data Access Object) design pattern. The system will allow users to add, view, and delete product information such as product ID, name, quantity, and price. Product data will be stored using

file handling instead of a database, making the application simple and lightweight. The use of the DAO pattern will separate data access logic from the application logic, improving maintainability, scalability, and code organization while demonstrating practical implementation of core Java programming concepts.

Target User

- Small shop owners and retail store managers
- Small businesses that need a simple inventory tracking system
- Students and beginner programmers learning Java OOP concepts
- Developers who want to understand the DAO design pattern with file storage

Core Features

- Add new products to the inventory
- View the list of available products
- Delete products from the inventory
- Store product information in a file for persistent storage
- Simple menu-driven console interface for user interaction
- Organized data management using Java classes and collections

OOP Concepts to be Used

Abstraction

- Implemented using the ProductDAO interface to define data operations without exposing implementation details.

Inheritance

- Future extensions can inherit base classes for expanding system functionality.

Polymorphism

- Achieved through implementing the ProductDAO interface with different DAO implementations.

Exception Handling

- Used to manage file input/output errors and prevent program crashes.

Collections / Threads (if applicable)

- Java Collections such as ArrayList will be used to store and manage product objects efficiently.

Proposed Architecture Description

The system will follow a layered architecture using the DAO design pattern. The Model layer contains the Product class representing product attributes such as ID, name, quantity, and price. The DAO layer includes the ProductDAO interface and ProductDAOImpl class that handle data operations like adding, retrieving, and deleting products. File handling will be used to store product data in a text file. The Main Application layer provides a console-based interface where users can interact with the system through menu options. This architecture separates data representation, data access, and user interaction, making the application more organized, maintainable, and scalable.

APPENDIX B — INSTRUCTOR REVIEW RESPONSE (APPROVED_MINOR)

From: Course Instructor To: Subhashree Das

Dear SUBHASHREE DAS,

Project: Inventory Management System using DAO Pattern + File Persistence Decision:

APPROVED_MINOR Score: 90

Strengths: The proposal clearly identifies a practical problem for small businesses and provides a well-structured solution suitable for beginner Java OOP students. The architectural description, utilizing the DAO pattern with clear separation of Model, DAO, and Main Application layers, is excellent and demonstrates a strong understanding of object-oriented design principles. The explicit mention of core OOP concepts to be applied (Abstraction, Polymorphism, Encapsulation, Exception Handling, Collections, Design Pattern) aligns perfectly with university learning objectives. The choice of file persistence over a database is appropriately justified for a lightweight, beginner-friendly system, emphasizing file handling as a key learning outcome. The scope is well-defined and manageable for a project of this nature.

Improvements:

1. Consider adding an 'Update Product' feature to provide full CRUD functionality, which is a common expectation for inventory management systems.
2. Elaborate on specific input validation and error handling strategies beyond just 'Exception Handling' - for instance, how the system will ensure unique product IDs, positive quantities, valid prices, and graceful handling of file I/O errors.
3. A 'Search Product' feature (e.g., by name or ID) would significantly enhance the system's usability.
4. Specify the intended format for the persistent data file (e.g., CSV, JSON, or a custom delimited format) for better clarity and to ensure robust data parsing and storage.
5. While the DAO pattern promotes scalability, the 'scalability' claim for a file-persistent, console-based system might be slightly overstated; it would be more accurate to emphasize 'maintainability' and 'extensibility' for future upgrades to a database.

Regards, Course Instructor

APPENDIX C — RESPONSE TO APPROVED_MINOR FEEDBACK

All five improvement points raised by the instructor were addressed in the final implementation:

1. Update Product: A full update dialog was implemented in ProductPanel, and ProductService.updateProduct() handles all validation for updates, providing complete CRUD functionality across all three entity types.
2. Validation strategies: ProductService implements six guard clauses covering empty names, non-positive prices, negative quantities and thresholds, and referential integrity checks for both categoryId and supplierId. FileManager wraps all I/O in try-with-resources with explicit RuntimeException rethrow for graceful I/O error handling.
3. Search Product: ProductPanel includes a live search field backed by ProductDAOImpl.searchByName() using Java Streams. A category filter combo-box provides additional drill-down capability without requiring a separate search operation.
4. Persistent data format: Java Object Serialization is used, producing binary .dat files (products.dat, categories.dat, suppliers.dat). This format is documented in the README. CSV export was also added as a human-readable alternative via CsvExporter.
5. Scalability vs. Maintainability: The report and README now correctly describe the benefit of the DAO pattern as "maintainability and extensibility", and the Future Scope section describes the precise steps needed to upgrade to a JDBC/database backend - demonstrating exactly the extensibility the pattern is designed to enable.
